# ZetaERP — Manual Bug-Testing SOP

## Part 9 — POS (Point of Sale): the billing terminal, sessions, payments, receipts, hold/recall

| | |
|---|---|
| **Document** | QA-SOP-09 — POS |
| **Product** | ZetaERP / Zenith (multi-tenant POS / ERP) |
| **Environment under test** | Production — https://erp.zetaclantech.com |
| **Scope of this part** | The **POS** module ( `/apps/zenith/pos/…` ): the **Billing Terminal** (product grid, cart, barcode scan, qty/price edit, GST tax toggle, sales-mode switching, customer attach, **Cash/Card/UPI + Split** payments, receipt auto-print, New Sale, **Hold/Void/Recall**, **bill edit / amend**, **credit / pay-later**), **Sessions** (open/close + cash reconciliation), **Terminals / Sales Modes / Product Prices** config, **Receipt** templates, **bill numbering**, **GST** on the bill, the **POS→Sales-invoice sync**, **offline mode**, reports/audit, and **multi-tenant isolation**. |
| **Audience** | Manual QA tester (no code knowledge assumed; basic GST literacy helps) |
| **Document owner** | Platform / Solo-dev |
| **Status** | Active |
| **Last updated** | 2026-07-02 |

## 1. Purpose

This SOP tells a tester **exactly how to manually test the POS terminal A→Z**, including edge and UI cases: open a session, ring up a sale (scan/tap products, edit qty/price, switch sales mode, toggle GST, attach a customer), take payment (cash/card/UPI and **split**), print the receipt, start a new sale, and **hold/recall/void/amend** bills — then verify the **money and GST are correct**, the **bill number is right and gap-free**, the **stock and Sales invoice sync**, and everything is **isolated per tenant**.

> **The POS is the live cash register.** A wrong total, a double-charge, a mis-numbered bill, or a cross-tenant leak is a real money/compliance problem. Do the arithmetic by hand, and **never test on MARUTHI's (or any live tenant's) real terminal, sessions, or bills.**

## 2. Critical rules for the tester (read before starting)

> ⚠️ **This is a LIVE cash register for a real shop (MARUTHI, org `100000`).**

1. **NEVER open a session, ring a sale, take a payment, void, hold, amend, or close a session** on the live flagship tenant **MARUTHI** or any live tenant's terminal. Use a **throwaway tenant** with its own terminal.
2. **Never alter or renumber an already-issued bill.** Editing a bill (amend) keeps the same number by design — do that only on throwaway bills.
3. **Never take a real payment.** Cash/Card/UPI here record against a test session; don't run a real card or UPI. The terminal does **not** hit a live payment gateway (payments are recorded, not charged), but still only use a throwaway session.
4. **Multi-tenant isolation is a top surface** (§14). Products, transactions, sessions, and terminals must be scoped to the active org; any cross-tenant read/write is **Critical**.
5. **GST on the bill is a legal record** — verify the CGST/SGST vs IGST split and the total by hand (§8). The POS→Sales-invoice sync makes the bill a tax invoice.
6. **Clean up** every test session/bill/terminal/product (and any throwaway tenant), or hand the dev the list. Don't leave an open test session or a PtCreated cart behind.
7. **Record the exact steps + amounts** for every case; for a money/GST case a bare "looks right" is not acceptable.

## 3. Test environment & preconditions

The POS is at `/apps/zenith/pos/<screen>` (lowercase `pos` ; the checkout is capital-**T** `Terminal` ). It renders on **/apps (Bootstrap)**; the rich optimistic terminal is **/apps-only** — **/qapps strips** `<script>` **/** `<style>` , so the Terminal client-side bounces `/qapps → /apps` . Test on **/apps**.

| Screen (URL) | Purpose |
|---|---|
| `/apps/zenith/pos` (→ Dashboard) | POS home / KPIs |
| `/apps/zenith/pos/Terminal` | **The billing terminal (checkout)** |
| `/apps/zenith/pos/PosSessions` (+ EditSession) | Open/close sessions, cash |
| `/apps/zenith/pos/PosTerminals` | Terminals config |
| `/apps/zenith/pos/PosTransactions` (+ EditTransaction) | Bill list (EditTransaction is legacy back-office — see §16) |
| `/apps/zenith/pos/CreditBills` | Pay-later bills |
| `/apps/zenith/pos/SalesMode` · `ProductPricing` | Sales modes · per-mode product prices |
| `/apps/zenith/pos/ReceiptConfig` | Receipt templates |
| `/apps/zenith/pos/Reports` · `BillNumberRegister` · `BillAuditReport` | Reports · bill register · amendment audit |
| `/apps/zenith/pos/OfflineMode` | PWA offline config + sync |

**Set up on the throwaway tenant:** a **terminal** (PosTerminals), a few **products** (some priced for Retail/Wholesale/Employee via Product Prices, some barcoded), a customer with a **GSTIN in the tenant's state** (intra) and one in **another state** (inter), the tenant's **GST config** (so the split works), and a **receipt template**. Optionally a real **barcode scanner** (it just types digits + Enter).

**Browsers:** full pass on Desktop Chrome; spot-check Edge/Safari/mobile/tablet. **Tools:** a **calculator** (verify GST + change), a **barcode scanner** (or keyboard), a phone for UPI/QR display checks, DevTools (Console/Network), and (optionally) **QZ Tray** for silent printing.

---

## 4. Test data

| Label | Use |
|---|---|
| Throwaway tenant + terminal | All POS testing |
| `PROD_18` / `PROD_5` | Products @18% / @5% GST with HSN + a scan code |
| `PROD_WS` | Product with a Wholesale price + a quantity-break threshold |
| `PROD_NOWS` | Product with **no** Wholesale price (should block in Wholesale mode) |
| `CUST_SAME` / `CUST_OTHER` | Customers with GSTIN in the tenant's / another state |
| Amounts | 1 × ₹100 @18% → taxable 100, GST 18 (intra CGST 9 + SGST 9), total 118 |

---

## 5. Conventions & severity

**Case Type:** `FUNC` · `GST` · `PAY` (payment) · `LIFE` (session/lifecycle) · `SCAN` · `EDGE` · `UI` · `A11Y` · `SEC` · `ISO` · `PERF`.

| Sev | Meaning | Example |
|---|---|---|
| **S1 — Critical** | Wrong total/change; double-charge; wrong or duplicate bill number; wrong GST on the issued bill; cross-tenant product/txn/session access; a completed sale lost | split total doesn't equal cash taken; two bills share a number; IGST on an intra-state sale |
| **S2 — High** | A checkout flow broken or clearly wrong | qty edit deletes the line; unpriced product billable in a fixed mode; receipt never prints; session won't close |
| **S3 — Medium** | Wrong/loose behaviour, limited impact | quantity-break doesn't trigger; tax starts in the wrong state on load; round-off off by a paisa |
| **S4 — Low** | Cosmetic / minor UX | grid tile spacing; mode-button colour; receipt alignment |

**Logging a defect** (template §16): Case ID, exact steps, **amounts (by-hand expected vs actual)**, tenant + terminal, session id, bill number, browser, screenshot (incl. the receipt), Console/Network. Evidence **mandatory** for GST, PAY, ISO, SEC.

---

## 6. Flow map (a sale, A→Z)

```
  Open Session (terminal + opening cash)
        │ PsOpen
        ▼
  BILLING  ── scan / tap product ─► CART (PtCreated) ── qty± / price-edit / remove
        │           ▲                    │ optimistic UI, syncs to Terminal/ajaxCart
        │           │ sales mode (store-wide + per-line), TAX on/off, attach customer
        ▼           │
  PAYMENT ── Cash F1 / Card F2 / UPI F3  OR  SPLIT (Cash+Card+UPI must sum ≥ payable)
        │  process#PosPayment (locked, no double-charge) → complete#PosTransaction
        ▼  mint POS/FY/NNNNN bill no. (race-safe, org+FY) · PtCompleted · async Sales invoice
  RECEIPT auto-print (QZ silent / hidden-iframe) ──► auto-advance to a fresh sale (~2.2s)
        │
  HOLD ⇄ RECALL (multi-bill) · VOID (cart only) · AMEND (edit issued bill, same no.) · CREDIT (pay later)
        ▼
  Close Session (closing cash → expected vs difference; discards any leftover cart)
```

---

# 7. THE TERMINAL — checkout A→Z

**Screen:** `/apps/zenith/pos/Terminal`. Sections: **No-Session** (open a session) → **Billing** (cart/grid/payment) → **Receipt** (post-sale "Sale Complete!" then auto-New-Sale).

## 7.1 Session + cart basics

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| PS-001 | LIFE | Open a session | Pick a terminal + opening cash → Open | Session opens (PsOpen); terminal warmed; billing screen shows | S2 |
| PS-002 | LIFE | One open session per terminal | Open a 2nd session on a terminal that already has one **with sales** | Blocked; an empty prior session is auto-closed instead | S2 |
| PS-003 | FUNC | Tap a product | Tap a grid tile | Line added to cart instantly (optimistic); total updates; cart is a PtCreated txn | S1 |
| PS-004 | FUNC | Category tabs | Click a category tab, then "ALL ITEMS" | Grid filters to that category / shows all (org's products) | S3 |
| PS-004a | PERF | **ALL ITEMS is instant** | From any category tab, click ALL ITEMS and immediately tap a tile | The full grid restores **instantly from cache** (never greyed/disabled, tiles tappable at once); a background refresh keeps it fresh (ZE-POS-0204 — used to freeze 5–10s on large catalogs) | S2 |
| PS-005 | FUNC | Product search | Type a name/code in the grid search | Grid filters (org-scoped) | S3 |
| PS-005a | FUNC | **Search-bar add = qty 1** | In the green cart search box type a name/code, pick a result (click or Enter) | Line added at **qty 1** (same as a grid tile), exactly ONE `ajaxCart` addItem POST (ZE-POS-0206 — the unguarded inline cart-search engine used to double-fire → qty 2, double-billed) | S1 |
| PS-005b | FUNC | Search-add shows the name | Add a product via the search box | Cart line shows the **product name** (code as secondary text), not the bare code twice (ZE-POS-0207 — search-add passed no name → optimistic line read `name‖pid` ) | S3 |
| PS-005c | FUNC | Search box still scans | Fire a physical barcode scanner into the search box | Fast entry (or no dropdown match) resolves the code server-side via `_posScan` (pseudoId/barcode) and adds once | S2 |
| PS-006 | FUNC | Qty + / − | Use +/− on a line | Qty and line total recompute; tax recomputes | S2 |
| PS-007 | EDGE | **Qty edit never deletes** | Set a line qty to 0 / empty / negative | Line is **not** deleted (reverts / floors); **only the × button removes** | S2 |
| PS-008 | FUNC | Price edit | Type a new unit price on a line | Line reprices; logged as a price override; line marked manual-price | S2 |
| PS-009 | EDGE | Negative price | Type a negative price | Rejected | S2 |
| PS-010 | FUNC | Remove line (×) | Click × on a line | Line removed; totals recompute | S3 |

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| PS-011 | EDGE | Empty-cart checkout | With an empty cart | Payment panel hidden / cannot complete a zero-item sale | S2 |

### 7.2 Barcode scan

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| PS-015 | SCAN | Scan a coded product | Scan into the cart search box | Correct product added instantly | S1 |
| PS-016 | SCAN | Unknown barcode | Scan a code with no product | Clear "Product not found"; nothing added | S2 |
| PS-017 | SCAN | Duplicate/ambiguous code | A code matching >1 product | Uses `list()` not `one()` → never silently mis-bills; resolves sensibly (see Part 7 Barcode) | S1 |
| PS-018 | SCAN | Scanner Enter | Real handheld (types digits + Enter) | Enter adds; no stray newline | S2 |

### 7.3 Sales modes, tax toggle, customer

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| PS-020 | FUNC | Store-wide mode switch | Switch Retail → Wholesale (mode buttons) | Whole cart reprices to Wholesale; **Wholesale requires a customer** + a min order | S2 |
| PS-021 | FUNC | Per-line mode | Change one line's mode via its dropdown; use "↻ Auto" to reset | That line reprices; a manually-set mode shows 🔒 (locked); Auto reverts to Retail + re-enables quantity-break | S2 |
| PS-022 | EDGE | **Unpriced product in a fixed mode** | Add `PROD_NOWS` in Wholesale (no wholesale price) | **Blocked** at add and at mode switch (a clear "no price for this mode" message) | S2 |
| PS-023 | FUNC | **Quantity-break** | Add `PROD_WS` at Retail; increase qty past its threshold | Line auto-switches to the Wholesale price at the threshold; reverts below; **no-op for products with no break configured** (MARUTHI-safe); a manual-price line is exempt | S2 |
| PS-024 | GST | **TAX toggle OFF/ON** | Toggle GST OFF then ON | OFF → all line tax zeroed + totals drop; ON → GST recomputed per line (exclusive). **[Fixed 80dd4b0 — a fresh session now shows "TAX: OFF" consistently (the UI default matched to the billing logic's OFF default); the button no longer falsely shows "ON". Toggle still works both ways.]** | S3 |
| PS-025 | FUNC | Attach a customer | Customer button → search (name/phone) → select | Customer attached to the open bill (name/phone/GSTIN); org-scoped search | S2 |
| PS-026 | SEC | Customer cross-org | Try to attach a customer from another tenant | Rejected (same-org owner check) | S1 |

### 7.4 GST correctness on the sale (do the math)

POS calls the India GST engine ( `calculate#Tax` ) at add/reprice/mode-change/tax-toggle; prices are **tax-exclusive**. Intra vs inter comes from the terminal's origin region vs the customer's.

| ID | Type | Title | Inputs | Expected result (by hand) | Sev |
|---|---|---|---|---|---|
| PS-030 | GST | Intra-state 18% | `CUST_SAME` , 1 × ₹100 @18% | taxable 100; **CGST 9 + SGST 9**; total 118 | S1 |
| PS-031 | GST | Inter-state 18% | `CUST_OTHER` , 1 × ₹100 @18% | taxable 100; **IGST 18**; total 118 | S1 |
| PS-032 | GST | Mixed rates | ₹100 @18% + ₹200 @5% | line1 GST 18, line2 GST 10; header tax 28; total 328 | S1 |
| PS-033 | GST | Round-off | A total like ₹117.90 | round-off applied to the payable (nearest step); receipt shows round-off | S2 |
| PS-034 | GST | Tax OFF sale | Toggle GST OFF, complete | Bill has 0 tax; the synced invoice is a **Bill of Supply** (tax 0), else a **Tax Invoice** | S2 |

### 7.5 Payments — Cash / Card / UPI + Split (do the math)

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| PS-040 | PAY | Cash exact (F1) | Cash, amount = payable | Sale completes; change 0; bill number minted | S1 |
| PS-041 | PAY | Cash over-payment | Cash > payable | **Change = paid − payable** shown/printed; sale completes | S1 |
| PS-042 | PAY | Card (F2) / UPI (F3) | Pay by Card / UPI | Records a PosPayment of that method; completes | S1 |
| PS-043 | PAY | **Split payment** | SPLIT → e.g. ₹50 Cash + ₹68 UPI on a ₹118 bill | Each >0 amount records its own PosPayment; completes; **sum must be ≥ payable** | S1 |
| PS-044 | PAY | **Split under total (rejected)** | Split amounts sum **< payable** | **Rejected** — sale does not complete | S1 |
| PS-045 | PAY | **No double-charge on retry** | Double-click Pay / re-submit after completing | The payment path is **locked/idempotent** — no second charge, no second bill | S1 |
| PS-046 | PAY | Over-payment change on split | Split summing > payable | Change computed correctly; but **non-cash (Card/UPI/etc.) tenders may NOT exceed payable** — only CASH can over-tender (change is returned from cash). Card/UPI over the bill total is **rejected** (ZE-POS-0208) | S1 |
| PS-047 | PAY | **Split non-cash over-tender (rejected)** | Split where Card+UPI (non-cash) sum **> payable**, e.g. UPI ₹5,000 + Cash ₹3,000 on a ₹200 bill | **Rejected** at complete#PosTransaction ("Card/UPI/non-cash payments exceed the bill total…") — atomic, no bill number minted; prevents un-returnable "change" corrupting session cash reconciliation (ZE-POS-0208) | S1 |
| PS-048 | UI | **Split inputs legible in dark mode** | Open SPLIT (F4) on Safari/Mac with OS dark mode; type amounts, unfocus | Cash/Card/UPI amount + reference inputs stay legible (white field / dark text via explicit `color` + `background` + `color-scheme:light`), not dark-on-dark (ZE-POS-0209) | S3 |

### 7.6 Complete → receipt → new sale

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| PS-050 | FUNC | Complete a sale | Pay | Status **PtCompleted**, `POS/FY/NNNNN` number minted at completion (not at cart start → no gaps); "Sale Complete!" summary | S1 |
| PS-051 | FUNC | **Receipt auto-prints** | After the sale | 80 mm receipt prints via **QZ Tray silent** (if connected) or a **hidden-iframe** fallback; correct items/tax/total/change | S1 |
| PS-052 | FUNC | Receipt never blocks a sale | Force a receipt error (e.g. no template) | The sale stays completed; response flags `receiptUnavailable` — **the completed sale is never lost/undone** | S1 |
| PS-053 | FUNC | **Auto-advance to new sale** | Wait ~2.2s after the sale | Auto-advances to a fresh empty sale; a **pointer-down cancels** the auto-advance; the summary does **not** loop back | S2 |
| PS-054 | FUNC | New Sale / F10 | Click New Sale (or F10) | Fresh empty cart; the previous completed bill is not carried forward | S2 |
| PS-055 | FUNC | Receipt PDF | Open a bill in **PosTransactions → EditTransaction** (back-office) and click **Download PDF**; also the post-sale summary's Download-PDF button | Downloads the server XSL-FO PDF ( `/fop/apps/zenith/pos/ReceiptPdf?transactionId=…` , filename `Receipt-<id>.pdf` ), same content as the printed receipt (items/qty/tax split/total/change/bill no.), using **this org's** receipt template (ZE-POS-0210) | S3 |

> **Note (ZE-POS-0210):** the ReceiptPdf XSL-FO screen existed but errored on every render (invalid FreeMarker `as number` ; missing `encode="false"` / `no-boundary-comment` ; missing `standalone="true"` so the POS chrome wrapped its `fo:root` ). Fixed, org-scoped the template to the bill's org, and wired a reliable **Download PDF** in EditTransaction (via `/fop` , like sales InvoicePrint) plus repointed the post-sale Terminal buttons `/apps` → `/fop` . The reliable path is EditTransaction (the post-sale summary auto-advances ~2.2s).

### 7.7 Hold / Recall / Void

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| PS-060 | LIFE | Hold a cart | Cart with items → HOLD (reason) | Cart snapshotted to a held bill; the terminal is freed for a new sale (transactionId cleared) | S2 |

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| PS-061 | LIFE | Recall | RECALL → pick a held bill | The held items return; **recall first auto-holds the current non-empty cart** (no merge/loss) | S2 |
| PS-062 | LIFE | Multi-bill hold | Hold two carts, recall each | Both preserved; switching between them loses nothing (native `<details>` picker) | S2 |
| PS-063 | LIFE | Void | Void a cart | PtCreated → **PtVoided** (cart only); can't void a completed bill; the terminal is freed | S2 |
| PS-064 | EDGE | Void/hold then new bill | After hold/void, add a product | A **fresh** cart starts (not "transaction already completed/voided") | S2 |

### 7.8 Bill edit (amend) & credit / pay-later

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| PS-070 | FUNC | **Amend an issued bill** | History → open an issued bill → edit items in the cart → re-bill | Re-bills the **same bill number**; stock adjusts by the **delta**; refund/balance reconciled; an **amendment audit** row is written | S1 |
| PS-071 | FUNC | Amend receipt reprint | After amend | The updated receipt reprints from the same receipt data | S2 |
| PS-072 | FUNC | Credit / pay-later | Complete a sale as **credit** | Bill created `isCredit='Y'`, paid 0; a per-phone customer party is created deterministically | S2 |
| PS-072a | FUNC | **Credit marker on the receipt** | Complete a credit sale → auto-print; also reprint from History and Download PDF | Every receipt path shows **"** CREDIT / PAY LATER (UNPAID) **"** + **Amount Due <outstanding>** (ZE-POS-005). Covers all 4 builders: server `_rd` / `printThermalReceipt`, JS `buildReceiptHtml` + `buildHistReceipt`, and the XSL-FO `ReceiptPdf`. Non-credit bills show none of it | S2 |
| PS-073 | PAY | Settle a credit bill | Credit Bills → the page **loads the outstanding dues list** (each row = customer + due), then settle | Page loads dues (NOT stuck on "Loading credit bills…"); settle records the payment via `Terminal/ajaxCart action=settleCredit`; `isCredit` clears when fully paid (ZE-POS-002: the page needs a rendered `moquiSessionToken` or the ajaxCart POST 401s and the list hangs) | S2 |
| PS-073a | FUNC | **Credit bill status in Transactions** | Transactions History → find a completed credit bill with a balance | Status reads **"Credit — Due <outstanding>"**, NOT "Completed" (the sale is `PtCompleted` but the receivable is open). Fully-settled/normal bills read "Completed"; voided read "Voided" (ZE-POS-004, display-only — underlying `statusId` unchanged) | S2 |

### 7.9 Role-based navigation — POS-only cashier (ZENITH_CASHIER)

A user whose **only** Zenith role is **ZENITH_CASHIER** (not the super-admin) is **till-first**: on login the ZenithApp gate lands them straight on the billing terminal (snappy — the /apps optimistic cart), but they **may step out to the Dashboard and Inventory** (create/manage products) and back. Every **other** module still bounces to the terminal. Test with a sole-ZENITH_CASHIER login **on the throwaway tenant** (never a live cashier).

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| PS-090 | FUNC | **Cashier lands in POS** | Log in as a sole-ZENITH_CASHIER user | Lands on `/apps/zenith/pos/Terminal` (the bare app root bounces here) — deliberate till-first landing, not an accidental redirect (ZE-POS-0201 S1) | S2 |
| PS-091 | FUNC | **Cashier can create items** | On the cashier's terminal header, click **+ ITEM** and **CATALOGUE** | Both **open** the Inventory product / catalogue screens (a new tab) and render — no 403, no bounce back to POS (ZE-POS-0201 Symptom 3; needs both the loosened gate AND the `ZENITH_CASHIER → Z_INVENTORY` authz grant) | S2 |
| PS-092 | FUNC | **Cashier reaches the Dashboard** | Click the header ⌂ **DASHBOARD** (or the brand logo); also the no-session "← Back to App" | Opens `/apps/zenith/Dashboard` (renders, not bounced). The cashier's Dashboard shows **only the POS + Inventory tiles** (every other module is hidden — clicking one would bounce). A ⏻ **LOGOUT** button gives a clean end-of-shift exit ( `/Login/logout` ) | S2 |
| PS-093 | GATE | **Other modules still bounce** | As the cashier, hit `/apps/zenith/Accounting/...` (or Sales/HR) by URL | Redirected back to `/apps/zenith/pos/Terminal` — only POS + Inventory + Dashboard are reachable | S2 |
| PS-094 | FUNC | **Non-cashier is unaffected** | Open the terminal as an **admin/manager** (broader role) | ITEM, CATALOGUE and ← **APP** (→ app dashboard) all show; no cashier-only ⌂ DASHBOARD / ⏻ LOGOUT buttons; the logo links to `/qapps/zenith`. The till-first trim applies to cashiers only | S2 |

# 8. SESSIONS, BILL NUMBERING & INVOICE SYNC

## 8.1 Sessions & cash

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| PS-080 | LIFE | Close session | Close with closing cash | `expected = opening + cash received − change given`; **cash difference** computed; leftover PtCreated cart discarded; idempotent (re-close is a no-op) | S2 |
| PS-081 | LIFE | Stale-cart cleanup | Open a cart, don't pay, re-open the session / start fresh | Abandoned PtCreated carts are retired at **session boundaries** — close#PosSession, open#PosSession (BOTH paths: start-fresh AND same-terminal empty-session auto-close, ZE-POS-007), and freshSale=Y — no old items reappear, no orphaned PtCreated rows on closed sessions. **There is deliberately NO time-based cart expiry**: within an open session the cart survives idle + refresh (a cashier mid-sale must not lose the cart); "cart still there after 5 min idle + refresh in the same session" is correct behaviour, not a defect. Safety net: nightly job `zenith_PosOrphanCartVoid_daily` (01:45, `void#OrphanedPosCarts`) voids any PtCreated cart stranded on a session closed >24h — never touches open sessions, numbered bills, or carts with recorded payments (those are WARN-logged for manual review) | S2 |
| PS-082 | LIFE | Cash entry / movement | Record a cash in/out | Reflected in the session cash reconciliation | S3 |
| PS-082a | VAL | **Cash / Expense / Wastage entry validation** | Cash Entry & Expense Entry: submit with Amount empty / `abc` / `0` / negative. Wastage Entry: Quantity empty / `0`, Unit Cost negative | **Rejected** with a clear message — no row saved (ZE-POS-008 + class sweep); valid values save normally (Wastage: `totalLoss = qty × unitCost`; unit cost 0 is allowed) | S3 |

## 8.2 Bill numbering (race-safe, org-scoped)

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| PS-085 | FUNC | Consecutive numbers | Complete several sales | `POS/FY/NNNNN` increments by 1, gap-free (minted at completion, not cart start) | S1 |
| PS-086 | PERF | **Concurrent terminals** | Two terminals complete sales at the same instant | **No duplicate/skipped** numbers (row-locked `PosBillSequence` per org+FY) | S1 |
| PS-087 | ISO | Per-tenant series | Compare numbers across two tenants | Each tenant has its own `org_FY` sequence — no cross-tenant sharing | S2 |

## 8.3 POS → Sales invoice sync

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| PS-090 | FUNC | Bill becomes an invoice | Complete a sale → check Sales | A finalized Sales invoice is created (async), **idempotent** (one per bill via `linkedInvoiceId`), using the POS bill number as the tax-invoice number | S2 |
| PS-090a | TAX | **Printed invoice matches the receipt (GST)** | GST sale → print the Tax Invoice (InvoicePrint) and compare to the receipt | Per-item **CGST/SGST/IGST + rates** shown; **Taxes = POS tax**, **Total = POS payable** (incl. round-off); amount-in-words matches the grand total; "reverse charge" renders Yes/No with **no inline template error**. A tax-OFF bill prints Taxes 0.00 / no GST columns even if stale TaxLineDetail rows exist (ZE-POS-009) | S2 |
| PS-091 | GST | Invoice GST matches | Compare the invoice's GST to the bill | Same CGST/SGST/IGST breakdown carried to the invoice items; **Tax Invoice if tax>0 else Bill of Supply** | S1 |
| PS-091a | UI | **Invoice barcode + clean layout** | Print any numbered invoice | Header shows a **scannable DataMatrix of the invoice number** + "Scan: <number>" caption (barcode4j FOP ext, ZE-POS-010) — no "[ QR CODE ]" placeholder, no raw template errors anywhere on the document; a number-less invoice omits the barcode block entirely | S3 |
| PS-092 | FUNC | Stock depletes | Sell an inventory-tracked product | Stock reduces on completion (only for `inventoryTracked='Y'` products) | S2 |

# 9. CONFIG — Terminals, Sales Modes, Product Prices, Receipts

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| PS-100 | FUNC | Terminal CRUD | PosTerminals → create/edit a terminal | Persists with `organizationPartyId`, GSTIN, UPI VPA, state | S3 |
| PS-101 | FUNC | Sales modes | SalesMode → inspect Retail/Wholesale/Employee | Retail (MRP, default, exclusive); Wholesale/Employee (fixed price, require customer) | S3 |
| PS-102 | FUNC | Product Prices bulk grid | ProductPricing → set per-mode prices + a quantity-break | Persists per `(product, mode)`; a fixed-mode product with no price is blocked at the till | S2 |
| PS-103 | FUNC | Receipt template | ReceiptConfig → edit company/header/footer/width (58/80mm), default | Reflected on the printed receipt; discount limit / round-off step honoured | S3 |

# 10. OFFLINE MODE (PWA) & REPORTS

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| PS-110 | FUNC | Offline config | OfflineMode → enable + export catalog | Catalog exported (**org-scoped**); PWA installable (manifest/sw) | S3 |
| PS-111 | EDGE | Offline sale sync | (Dev-assisted) queue an offline sale, sync | Idempotent via `OFFLINE:<id>` payment ref — **no double-post** on retry | S2 |
| PS-111a | FUNC | **Bill History while offline** | Go offline → make offline sales → open Bill History (F8) | Shows the **bills saved on this device** ("OFFLINE — PENDING SYNC" rows: amount, time, items, method, with the pending-sync banner), NOT a bare connection error; search filters by amount/item/method; error message only when nothing is queued; back online the normal server history returns (ZE-POS-017) | S2 |
| PS-112 | FUNC | Reports | Reports → Bill-wise / Item-wise / Payment-wise / User-wise / Cancelled | Org-scoped totals reconcile to the session/bills | S3 |
| PS-113 | FUNC | Bill register / audit | BillNumberRegister; BillAuditReport | Consecutive numbers listed; amendment audit shows who changed what | S2 |

# 11. MULTI-TENANT ISOLATION (high priority)

`PosTransaction / PosSession / PosTerminal` (and PosExpense, PosCashEntry, price overrides, KOT, QR) are org-filtered (`MANTLE_ACTIVE_ORG`). The terminal scopes the grid, categories, customer search, catalog export, and scan by org.

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| ISO-001 | ISO | Grid/transactions scoped | Cashier on tenant A | Only A's products, bills, sessions, terminals | S1 |
| ISO-002 | ISO | Same barcode two tenants | A and B each have a product with the same code | Each till resolves its own product | S1 |
| ISO-003 | SEC | Direct-URL to another tenant's bill/session | Open `EditTransaction` / `EditSession` with B's id | Blocked / not found | S1 |
| ISO-004 | ISO | Catalog export / offline sync | A exports catalog / syncs | Only A's products; sync can't post to B | S1 |
| ISO-005 | SEC | **Child entities via a direct screen** | Any screen listing `PosPayment` / `PosTransactionItem` / `PosHeldTransaction` / `PosProductModePrice` directly | These have **no direct org filter** (reached only via their org-scoped parent) — verify no direct-entity screen leaks them cross-tenant | S2 |

# 12. UI / ACCESSIBILITY / SECURITY

| ID | Type | Title | Check | Expected result | Sev |
|---|---|---|---|---|---|
| PS-120 | UI | Terminal on a real till screen | 1366×768 / 1080p / a small 10" tablet | Grid + cart + payment fit; no clipping; big tap targets | S3 |
| PS-121 | UI | **/qapps bounces to /apps** | Open `/qapps/zenith/pos/Terminal` | Redirects to `/apps/...` (the rich terminal is /apps-only) | S2 |
| PS-121a | NAV | **Dashboard "Manage" shortcut tiles open** | Dashboard → click each Manage tile (Expenses, Cash Entry, Stock, Wastage, Kitchen, Import, Sales Mode, Receipts, Sessions, Terminals, Price Overrides, Credit Bills) | Each opens its screen (NOT a 404). Tiles link to the absolute `/apps/zenith/pos/<Screen>` — a raw `<a href="goX">` transition name resolves to `.../pos/goX` (sibling) → 404 on both surfaces (ZE-POS-003) | S2 |
| PS-122 | A11Y | Keyboard shortcuts | F1 (Cash) / F2 (Card) / F3 (UPI) / F10 (New Sale) | Fire the right action; visible focus | S3 |
| PS-123 | SEC | XSS in product/customer name | Product/customer name `<img src=x onerror=alert(1)>` | Escaped in the cart, grid, receipt, and history; never executes | S1 |
| PS-124 | SEC | Transitions need the session token | (Dev) POST a cart/pay transition without the token | Rejected (offline export/sync are deliberately token-free — verify those are still org-scoped) | S2 |

## 13. Cross-flow / regression & the double-engine watch

| ID | Type | Title | Expected result | Sev |
|---|---|---|---|---|
| XF-001 | FUNC | **Single cart engine** | The cart updates **once** per action — no flicker/double-line/double-POST. (There is a legacy inline engine that must stand down to the external `pos-terminal.js`.) Watch the Network tab for duplicate `ajaxCart` posts | S1 |
| XF-002 | GST | POS bill ↔ Sales invoice ↔ Tax engine | The bill's GST, the synced invoice's GST, and what the Tax engine (Part 6) computes all agree | S1 |
| XF-003 | LIFE | Full day | Open → several sales (cash/card/upi/split/credit/hold/recall/amend) → close; the session cash reconciles and bill numbers are gap-free | S1 |
| XF-004 | ISO | Two-tenant | Nothing tenant A rings up is visible to tenant B | S1 |

```
Defect ID:        ZE-POS-____
Linked Case ID:   PS-0__ / ISO-0__ / XF-0__
Title / Severity / Type:
Environment:      Prod | <browser + scanner/printer> | tenant: <org> · terminal: <id> (NEVER MARUTHI)
Steps + amounts:  cart lines, mode, tax on/off, payment method(s) + amounts
Expected (by hand): taxable, CGST/SGST/IGST, round-off, payable, change, bill no.
Actual:           (same) — attach screenshot + the receipt + Console/Network
Session/bill ids:
```

## 14. Areas to probe hardest (attention map)

1. **Payment integrity (PS-043/044/045)** — split must sum to ≥ payable; no double-charge on retry/double-click; change is exact. This is the money.
2. **Bill numbering under concurrency (PS-086)** — two terminals at once must never duplicate or skip a number; per org+FY.
3. **GST split on the bill *and* the synced invoice (PS-030/031/091, XF-002)** — CGST+SGST intra vs IGST inter, and the invoice matches the bill and the tax engine.
4. **The cart double-engine (XF-001)** — verify only the external `pos-terminal.js` runs (the legacy inline engine stands down); watch for double `ajaxCart` posts / flicker / double lines.
5. **Qty edit never deletes (PS-007)** and **unpriced-product-in-fixed-mode block (PS-022)** — common till mistakes.
6. **Hold/Recall/Void carry-forward (PS-060..064)** — after hold/void the next add must start a **fresh** cart (not target the held/voided one); recall auto-holds the current cart.
7. **Amend an issued bill (PS-070)** — same number, stock delta, reconciled refund/balance, audit row — never a silent renumber or a stock double-count.
8. **Receipt never undoes a completed sale (PS-052)** — a print failure must leave the sale completed.

9. **Tax default state on load (PS-024) — fixed (80dd4b0).** The UI now defaults to the same OFF state as the billing logic, so a fresh session shows "TAX: OFF" honestly; re-verify the button matches the computed tax and that the toggle works both ways.
10. **Multi-tenant isolation (§11)** — especially the child entities (payments/items/held/mode-price) that rely on their parent's org scope; and that offline export/sync (token-free) stay org-scoped.

---

## 15. Teardown (mandatory)

1. **List every test artefact** — sessions (close them), bills, held bills, credit bills, amendments, terminals, products, receipt templates, offline queue — with ids.
2. **Close any open test session** and discard leftover PtCreated carts; don't leave an open session on a terminal.
3. **Void/delete** test bills (or hand the dev the list); never touch a real bill's number.
4. Delete test products/terminals/templates and any throwaway tenant (see the Auth/Signup teardown recipe).
5. Confirm you **never touched MARUTHI** (org `100000`) or any live tenant's terminal, sessions, bills, or payments.
6. Attach the run sheet(s) + defect ids to this part's report.

---

## 16. Known dead / fragile spots (for the tester's awareness)

- **`EditTransaction.xml`** (the `/apps/zenith/pos/PosTransactions → EditTransaction` back-office bill screen) **works** — a re-check corrected the earlier recon: the GST fields ( `customerGstin` , `isB2b` , `cgstTotal/sgstTotal/igstTotal` ) are declared on `PosTransaction` via the **`zenith-erp-india`** extension ( `IndiaExtendEntities.xml` ), and `generate#UpiQrString` exists in that same component ( `IndiaPosServices.xml` ); the screen renders HTTP 200 and derives the GST split for display. It is a legacy back-office **bill viewer / draft-cart editor** (hidden from the menu); the primary live bill-edit path is still the Terminal's amend flow (PS-070), but this screen is not broken.
- **`.bak` files** ( `Terminal.xml.bak` , `ReceiptPdf.xml.bak` , `PosServices.xml.bak` ) are stale copies, not deployed — ignore.
- **/qapps** strips `<script>` / `<style>` — the rich terminal only works on **/apps** (verify the /qapps→/apps bounce, PS-121).

---

*End of Part 9 — POS. Related: Part 6 — Tax (the GST engine the till uses); Part 4 — Sales (the invoice the bill syncs to); Part 7 — Barcode (scan resolution); Part 2 — Contacts (the customer attached). The highest-risk areas to raise with the dev after a run: any duplicate/skipped bill number under concurrency, any double-charge or split-total mismatch, any GST mismatch between the bill and its synced invoice, and any cart double-engine flicker/double-post.*